

Lab1 Report

Part 1: PC Bootstrap

Exercise 1

Familiarize yourself with the assembly language materials available on the 6.828 reference page. You don't have to read them now, but you'll almost certainly want to refer to some of this material when reading and writing x86 assembly.

We do recommend reading the section "The Syntax" in Brennan's Guide to Inline Assembly. It gives a good (and quite brief) description of the AT&T assembly syntax we'll be using with the GNU assembler in JOS.

AT&T汇编语法 https://www.delorie.com/djgpp/doc/brennan/brennan_att_inline_djgpp.html

1. 寄存器使用%: `%eax`
2. 目的寄存器总是在左边
3. 立即数和常数使用\$: `$0xd00d`
4. `b`(byte), `w`(word), `l`(longword): `movw`
5. 访存`immed32(basepointer, indexpointer, indexscale)`的地址为`immed32 + basepointer + indexpointer * indexscale`

[Intel Architecture Manual](#)

Exercise 2

Use GDB's `si` (Step Instruction) command to trace into the ROM BIOS for a few more instructions, and try to guess what it might be doing. You might want to look at Phil Storrs I/O Ports Description, as well as other materials on the 6.828 reference materials page. No need to figure out all the details - just the general idea of what the BIOS is doing first.

```
(shell 1) ~/oslab/lab/$ make qemu-gdb
(shell 2) ~/oslab/lab/$ make gdb
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
This GDB was configured as "x86_64-linux-gnu".
+ target remote localhost:26000
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
warning: A handler for the OS ABI "GNU/Linux" is not built into this configuration
of GDB. Attempting to continue with the default i8086 settings.

The target architecture is set to "i8086".
[f000:fff0] 0xfffff0: ljmp  $0xf000,$0xe05b
0x0000fff0 in ?? ()
```

这是 GDB 从远程目标 (QEMU) 读到的第一条指令，下面将逐项解释：

```
[f000:fff0]    0xffff0: ljmp    $0xf000,$0xe05b
0x0000fff0 in ?? ()
```

CS - Code Segment: CS是代码段寄存器。在实模式下，它的值不是一个直接的内存地址，而是一个指向内存中某个段（Segment）的段基址（Segment Base Address）的指针。

IP - Instruction Pointer: IP是指令指针寄存器（在32位模式下被称为EIP，64位下为RIP）。它可以看作是当前代码段（CS指向的段）内的一个偏移量（Offset）。

```
[f000:fff0] -> CS = 0xf000 and IP = 0xffff0
physical address = 16 * segment + offset
                  = 0xf0000 + 0xffff0
                  = 0xffff0 -> 16 bytes before the end of the BIOS (0x100000)

0xffff0: ljmp    $0xf000,$0xe05b -> jmp backwards to an earlier location in the
BIOS
-> jumps to the segmented address CS = 0xf000 and IP = 0xe05b
-> physical address = 0xfe05b
-> 跳转到一个更低、更宽敞的地址 (0xfe05b)，那里有足够的连续空间来存放 BIOS 需要执行的大量初始化代码（检测硬件、建立中断向量表、扫描引导设备等）

0x0000fff0 in ?? () -> 这是GDB对当前程序计数器PC - Program Counter) 位置的另一种表示。因为此时还没有加载任何符号信息，GDB不知道这个地址对应什么函数，所以显示??
```

Run Step Instruction command in gdb to see what happens next:

```
(gdb) si
[f000:e05b]    0xfe05b: cml    $0x0,%cs:0x6ac8
0x0000e05b in ?? ()
(gdb) si
[f000:e062]    0xfe062: jne    0xfd2e1
0x0000e062 in ?? ()
(gdb) si
[f000:e066]    0xfe066: xor     %dx,%dx
0x0000e066 in ?? ()
(gdb) si
[f000:e068]    0xfe068: mov     %dx,%ss
0x0000e068 in ?? ()
(gdb) si
[f000:e06a]    0xfe06a: mov     $0x7000,%esp
0x0000e06a in ?? ()
```

```
cml    $0x0,%cs:0x6ac8 -> 比较物理地址0xf6ac8处的4字节值是否为0，可能在初始化检查
jne    0xfd2e1 -> 如果不为0，跳转到0xfd2e1
xor     %dx,%dx -> %dx置零
mov     %dx,%ss -> %ss置零，%ss是栈段寄存器(Stack Segment Register)，栈段的基地址被设
```

置为 0x000000

mov \$0x7000,%esp -> %esp置为0x7000, %esp扩展栈指针寄存器(Extended Stack Pointer)

-> 当前的栈顶物理地址为: $(SS \ll 4) + SP = (0x0000 \ll 4) + 0x7000 = 0x07000$

-> 在物理内存的低地址区 (0x07000) 建立了一个栈, 任何push操作都会将数据写入低于0x07000的内存中

Part 2: The Boot Loader

Exercise 3

Take a look at the lab tools guide, especially the section on GDB commands. Even if you're familiar with GDB, this includes some esoteric GDB commands that are useful for OS work.

Set a breakpoint at address 0x7c00, which is where the boot sector will be loaded. Continue execution until that breakpoint. Trace through the code in boot/boot.S, using the source code and the disassembly file obj/boot/boot.asm to keep track of where you are. Also use the x/i command in GDB to disassemble sequences of instructions in the boot loader, and compare the original boot loader source code with both the disassembly in obj/boot/boot.asm and GDB.

Trace into bootmain() in boot/main.c, and then into readsect(). Identify the exact assembly instructions that correspond to each of the statements in readsect(). Trace through the rest of readsect() and back out into bootmain(), and identify the begin and end of the for loop that reads the remaining sectors of the kernel from the disk. Find out what code will run when the loop is finished, set a breakpoint there, and continue to that breakpoint. Then step through the remainder of the boot loader.

```
(gdb) b *0x7c00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.
[ 0:7c00] => 0x7c00: cli
Breakpoint 1, 0x00007c00 in ?? ()
(gdb) si
[ 0:7c01] => 0x7c01: cld
0x00007c01 in ?? ()
(gdb) si
[ 0:7c02] => 0x7c02: xor    %ax,%ax
0x00007c02 in ?? ()
```

-> 后面就是boot/boot.S里的汇编, 说明BIOS已经将boot loader加载到0x7c00处并跳转到这里让boot loader take over了, 可以对照着boot/boot.S和obj/boot/boot.asm来看

Questions

1. At what point does the processor start executing **32-bit code**? What exactly causes the switch from 16- to 32-bit mode?
2. What is the **last instruction of the boot loader** executed, and what is the **first instruction of the kernel** it just loaded?
3. Where is the **first instruction of the kernel**?

4. How does the boot loader decide **how many sectors** it must read in order to fetch the entire kernel from disk? Where does it find this information?

Analysis

`boot loader`指的是BIOS加载`kernel`的程序，反汇编在`obj/boot/boot.asm`中

由`boot/boot.S`和`boot/main.c`组成，目的是调用`kernel`和此前的准备工作（转换地址翻译模式等）

`bootloader (boot.S) -> bootmain (main.c)`

1. 加载全局描述符表寄存器(GDTR) `gdt_desc`标签处的数据包含了GDT的大小和起始地址，这条指令让CPU知道了我们定义的保护模式段表在哪里

```
%cr0 = $CR0_PE_ON = 0x1 (protected mode enable flag)
```

正式进入保护模式，但此时的内存寻址方式尚未更新，因为CS还是实模式的值

`boot/boot.S`

```
# Switch from real to protected mode, using a bootstrap GDT
# and segment translation that makes virtual addresses
# identical to their physical addresses, so that the
# effective memory map does not change during the switch.
lgdt    gdt_desc
movl    %cr0, %eax
orl     $CR0_PE_ON, %eax
movl    %eax, %cr0
```

2. 将`$PROT_MODE_CSEG(0x8)`加载到CS寄存器。这使CS现在作为一个选择子(index?)，指向GDT中定义的内核代码段

跳转到`protcseg`标签处的指令继续执行

彻底完成了到保护模式的切换，因为它更新了CS，CPU从此开始使用GDT中的描述符来翻译地址

```
# Jump to next instruction, but in 32-bit code segment.
# Switches processor into 32-bit mode.
ljmp    $PROT_MODE_CSEG, $protcseg

.code32                                # Assemble for 32-bit mode
```

`obj/boot/boot.asm`

```
7c1e:  0f 01 16          lgdtl  (%esi)
7c21:  64 7c 0f          fs  jl  7c33 <protcseg+0x1>
7c24:  20 c0             and    %al,%al
7c26:  66 83 c8 01       or     $0x1,%ax
7c2a:  0f 22 c0          mov    %eax,%cr0
```

```

7c2d:  ea                      .byte 0xea
7c2e:  32 7c 08 00             xor    0x0(%eax,%ecx,1),%bh

```

重置段指针后调用`bootmain`

`bootmain (main.c) -> kernel (kernel.asm)`

1. 从硬盘的第0扇区开始 (`offset = 0`)，读取4KB (`SECTSIZE*8 = 512*8 = 4096`) 的数据到内存地址 `0x10000`

`boot/main.c`

```

`#define ELFHDR      ((struct Elf *) 0x10000) // scratch space`

// read 1st page off disk
readseg((uint32_t) ELFHDR, SECTSIZE*8, 0);

```

2. 检查内存中ELFHDR位置的`e_magic`字段是否等于标准的ELF魔数

```

if (ELFHDR->e_magic != ELF_MAGIC)
    goto bad;

```

3. 遍历程序头表中的每一个条目，对于每个段：

`ph->p_pa`: 物理地址，该段应该被加载到的目标内存地址

`ph->p_memsz`: 该段在内存中的大小

`ph->p_offset`: 该段在ELF文件中的起始偏移量

程序头表是一个由多个`Proghdr`（程序头）结构组成的数组。每个`Proghdr`描述了一个内核中的段（**Segment**），例如代码段（`.text`）、数据段（`.data`）等。要遍历程序头表，首先要知道程序头表的位置和大小

`eph = ph + ELFHDR->e_phnum`; 是终止循环的判断，所以`boot loader`是通过ELF文件头中的信息得知一些数目信息的。`Boot Loader`首先从磁盘的第一个扇区（紧跟在`boot loader`自身之后）读取足够的数据到内存中的`0x10000`（见1.）。这部分数据包含了内核映像的开头，其中就包括ELF文件头

ELF文件头就像一个文件的“总目录”，它本身并不包含代码数据，但它指明了在哪里可以找到真正的“地图”。其中两个关键字段是：

`e_phoff`: 程序头表（**Program Header Table**）在文件中的偏移量（字节为单位）

`e_phnum`: 程序头表中一共有多少个条目

通过这两个值，`boot loader`就能找到并遍历程序头表

对于每一个段，`Proghdr`提供了将其从磁盘加载到内存所需的全部信息：`ph->p_pa`，`ph->p_memsz`，`ph->p_offset`

ELF -> Proghdr -> Section (.text, .rodata, .data, .bss)

```
// load each program segment (ignores ph flags)
ph = (struct Proghdr *) ((uint8_t *) ELFHDR + ELFHDR->e_phoff);
eph = ph + ELFHDR->e_phnum;
for (; ph < eph; ph++)
    readseg(ph->p_pa, ph->p_memsz, ph->p_offset);
```

`Boot Loader`决定读取多少扇区的过程：先读一点（ELF头）：读取磁盘开头的少量扇区，获取ELF头；根据ELF头提供的信息，找到程序头表；遍历程序头表的每一个条目，每个条目告诉`boot loader`一个段的大小和位置。`Boot Loader`于是读取每个segment (`readseg`)所在的sector (`readsect`)。

4. 跳转到内核入口

`ELFHDR->e_entry`是ELF头中指定的内核的入口点地址（一个内存地址）

这行代码将该地址强制转换为一个无参数、无返回值的函数指针，然后调用这个函数

这是`boot loader`和`kernel`的转换点

```
((void (*)(void)) (ELFHDR->e_entry))();
```

`bad`让程序挂起

```
bad:
    outw(0x8A00, 0x8A00);
    outw(0x8A00, 0x8E00);
    while (1)
        /* do nothing */;
```

对应的反汇编代码：

obj/boot/boot.asm

```
7d58: 73 17                                jae    7d71 <bootmain+0x58>
7d71: ff 15 18 00 01 00                call   *0x10018
    -> 循环条件不满足时跳出，跳转到kernel入口
    -> 后面由boot.asm转到kernel.asm
```

```
(gdb) b *0x7d71
Breakpoint 3 at 0x7d71
```

```
(gdb) c
Continuing.
=> 0x7d71:      call    *0x10018
    -> boot loader执行的最后一条指令
Breakpoint 3, 0x00007d71 in ?? ()
(gdb) si
=> 0x10000c:     movw    $0x1234,0x472
    -> kernel执行的第一条指令
    -> 对应kernel.asm中
    -> f010000c:    66 c7 05 72 04 00 00      movw    $0x1234,0x472
0x0010000c in ?? ()
```

Exercise 4

Read about programming with pointers in C. The best reference for the C language is The C Programming Language by Brian Kernighan and Dennis Ritchie (known as 'K&R'). We recommend that students purchase this book (here is an Amazon Link) or find one of MIT's 7 copies.

Read 5.1 (Pointers and Addresses) through 5.5 (Character Pointers and Functions) in K&R. Then download the code for pointers.c, run it, and make sure you understand where all of the printed values come from. In particular, make sure you understand where the pointer addresses in printed lines 1 and 6 come from, how all the values in printed lines 2 through 4 get there, and why the values printed in line 5 are seemingly corrupted.

There are other references on pointers in C (e.g., A tutorial by Ted Jensen that cites K&R heavily), though not as strongly recommended.

Warning: Unless you are already thoroughly versed in C, do not skip or even skim this reading exercise. If you do not really understand pointers in C, you will suffer untold pain and misery in subsequent labs, and then eventually come to understand them the hard way. Trust us; you don't want to find out what "the hard way" is.

```
~/oslab$ gcc -g -o pointers pointers.c
~/oslab$ ./pointers
1: a = 0x7ffdfd603680, b = 0x5c6403c702a0, c = (nil)
2: a[0] = 200, a[1] = 101, a[2] = 102, a[3] = 103
3: a[0] = 200, a[1] = 300, a[2] = 301, a[3] = 302
4: a[0] = 200, a[1] = 400, a[2] = 301, a[3] = 302
5: a[0] = 200, a[1] = 128144, a[2] = 256, a[3] = 302
6: a = 0x7ffdfd603680, b = 0x7ffdfd603684, c = 0x7ffdfd603681
```

Exercise 5

Trace through the first few instructions of the boot loader again and identify the first instruction that would "break" or otherwise do the wrong thing if you were to get the boot loader's link address wrong. Then change the link address in boot/Makefrag to something wrong, run make clean, recompile the lab with make, and trace into the boot loader again to see what happens. Don't forget to change the link address back and make clean again afterward!

boot/Makefrag

```
$(V)$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o $@.out $^
```

```
-> $(V)$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C01 -o $@.out $^
```

```
(gdb) b *0x7c00
Breakpoint 1 at 0x7c00
(gdb) c
Continuing.
[ 0:7c00] => 0x7c00: xchg %eax,%eax

Breakpoint 1, 0x00007c00 in ?? ()
(gdb) si
[ 0:7c02] => 0x7c02: nop
0x00007c02 in ?? ()
(gdb) si
[ 0:7c03] => 0x7c03: cli <- boot sector
0x00007c03 in ?? ()
(gdb) si
[ 0:7c04] => 0x7c04: cld
0x00007c04 in ?? ()
(gdb) si
[ 0:7c05] => 0x7c05: xor %ax,%ax
0x00007c05 in ?? ()
(gdb) si
[ 0:7c07] => 0x7c07: mov %ax,%ds
```

start-0x3处的两条指令是由链接器插入的填充或对齐指令，所以boot sector在0x7c04处开始

obj/boot/boot.asm

```
00007c01 <start-0x3>:
    7c01: 66 90                xchg %ax,%ax
    7c03: 90                   nop

00007c04 <start>:
    7c04: fa                   cli
    7c05: fc                   cld
    7c06: 31 c0                xor %eax,%eax
    7c08: 8e d8                mov %eax,%ds
    7c0a: 8e c0                mov %eax,%es
    7c0c: 8e d0                mov %eax,%ss
```

Load Address和Link Address不一致导致

1. BIOS将引导扇区的512字节数据从磁盘加载到物理内存地址0x7c00到0x7dff的区域

2. BIOS执行`jmp 0x7c00`，将CPU的控制权交给`boot loader`的第一个字节
3. 链接器在解析代码中的所有绝对地址时，都基于一个假设：代码的起始点在`0x7c04`
4. 绝对地址引用导致崩溃

Exercise 6

We can examine memory using GDB's `x` command. The GDB manual has full details, but for now, it is enough to know that the command `x/Nx ADDR` prints `N` words of memory at `ADDR`. (Note that both 'x's in the command are lowercase.) Warning: The size of a word is not a universal standard. In GNU assembly, a word is two bytes (the 'w' in `xorw`, which stands for word, means 2 bytes).

Reset the machine (exit QEMU/GDB and start them again). Examine the 8 words of memory at `0x00100000` at the point the BIOS enters the boot loader, and then again at the point the boot loader enters the kernel. Why are they different? What is there at the second breakpoint? (You do not really need to use QEMU to answer this question. Just think.)

在BIOS进入`boot loader`时，物理地址`0x00100000`的内容是未初始化的内存。而在`boot loader`进入内核时，该地址的内容已经变成了内核的`.text`代码段的前几个字（见上面的`Program Header`输出条目）。地址`0x00100000`的内容在`boot loader`调用`bootmain()`函数之后，执行`readseg()`函数的过程中被改变的。

```
(gdb) x/8x 0x100000
0x100000:      0x00000000      0x00000000      0x00000000      0x00000000
0x100010:      0x00000000      0x00000000      0x00000000      0x00000000

(gdb) x/8x 0x100000
0x100000:      0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:      0x34000004      0xd000b812      0x220f0010      0xc0200fd8
```

```
kern/kernel.ld

/* Link the kernel at this address: "." means the current address */
. = 0xF0100000;

/* AT(...) gives the load address of this section, which tells
   the boot loader where to load the kernel in physical memory */
.text : AT(0x100000) {
    *(.text .stub .text.* .gnu.linkonce.t.*)
}
```

Operating system kernels linked and run at very high virtual address, such as `0xf0100000`

in order to leave the lower part of the processor's virtual address space for user programs to use

map virtual address `0xf0100000` (the link address at which the kernel code expects to run) to physical address `0x00100000` (where the boot loader loaded the kernel into physical memory)

Part 3: The Kernel

Exercise 7

Use QEMU and GDB to trace into the JOS kernel and stop at the `movl %eax, %cr0`. Examine memory at `0x00100000` and at `0xf0100000`. Now, single step over that instruction using the `stepi` GDB command. Again, examine memory at `0x00100000` and at `0xf0100000`. Make sure you understand what just happened.

What is the first instruction after the new mapping is established that would fail to work properly if the mapping weren't in place? Comment out the `movl %eax, %cr0` in `kern/entry.S`, trace into it, and see if you were right.

```
obj/kern/kernel.san

f0100025:  0f 22 c0          mov    %eax,%cr0
```

在执行这一条指令之后，新的mapping已经建立了，`0xf0100000`会被映射到`0x00100000`

```
(gdb) b *0x0100025
Breakpoint 2 at 0x100025
(gdb) c
Continuing.
=> 0x100025:  mov    %eax,%cr0

Breakpoint 2, 0x00100025 in ?? ()
(gdb) x/8x 0x00100000
0x100000:  0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:  0x34000004      0xd000b812      0x220f0010      0xc0200fd8
(gdb) x/8x 0xf0100000
0xf0100000 <_start-268435468>:  0x00000000      0x00000000      0x00000000      0x00000000
0x00000000
0xf0100010 <entry+4>:  0x00000000      0x00000000      0x00000000      0x00000000
(gdb) si
=> 0x100028:  mov    $0xf010002f,%eax
0x00100028 in ?? ()
(gdb) x/8x 0x00100000
0x100000:  0x1badb002      0x00000000      0xe4524ffe      0x7205c766
0x100010:  0x34000004      0xd000b812      0x220f0010      0xc0200fd8
(gdb) x/8x 0xf0100000
0xf0100000 <_start-268435468>:  0x1badb002      0x00000000      0xe4524ffe
0x7205c766
0xf0100010 <entry+4>:  0x34000004      0xd000b812      0x220f0010      0xc0200fd8
```

这是建立映射后的第一条指令

```
mov $relocated, %eax
```

注释`kern/entry.S`中的`movl %eax, %cr0`后

```

(gdb) b *0x100020
Breakpoint 1 at 0x100020
(gdb) c
Continuing.
The target architecture is set to "i386".
=> 0x100020:    or     $0x80010001,%eax

Breakpoint 1, 0x00100020 in ?? ()
(gdb) si
=> 0x100025:    mov     $0xf010002c,%eax <- first instruction
0x00100025 in ?? ()
(gdb) si
=> 0x10002a:    jmp     *%eax
0x0010002a in ?? ()
(gdb) si
=> 0xf010002c <relocated>:    add     %al,(%eax) <- fail to work
0xf010002c in relocated ()
(gdb) si
Remote connection closed

```

因为此时没有建立正确映射，导致链接地址被当作物理地址访问

```
0xf010002c <relocated>: 0x00000000    0x00000000    0x00000000    0x00000000
```

对应加载出来的错误指令 `add %al,(%eax)`，因为它的机器码为 `00 00`

Exercise 8

We have omitted a small fragment of code - the code necessary to print octal numbers using patterns of the form `"%o"`. Find and fill in this code fragment.

```

case 'o':
    // Replace this with your code.
    num = getuint(&ap, lflag);
    base = 8;
    goto number;
break;

```

Questions

1. Explain the interface between `printf.c` and `console.c`. Specifically, what function does `console.c` export? How is this function used by `printf.c`?

下面看 `cprintf` 的实现

```

kern/print.c

int
cprintf(const char *fmt, ...)
{
    va_list ap;           // variable arguments
    int cnt;

    va_start(ap, fmt);    // 取出参数到ap中, 并指定...之前的参数
    cnt = vfprintf(fmt, ap); // 将取出的参数列表传给真正的实现函数
    va_end(ap);           // 释放参数列表

    return cnt;
}

int
vcprintf(const char *fmt, va_list ap)
{
    int cnt = 0;

    vprintfmt((void*)putch, &cnt, fmt, ap);
    return cnt;
}

```

`printf.c`调用了`printfmt.c`中的`vprintfmt`，它的一个参数为`printf.c`中的`putch`函数，而`putch`函数又调用了`console.c`中的`cputchar`函数。

所以`cputchar`是`console.c`暴露的接口。

```
cprintf -> vcprintf -> vprintfmt -> putch -> cputchar
```

2. Explain the following from `console.c`:

```

1     if (crt_pos >= CRT_SIZE) {
2         int i;
3         memmove(crt_buf, crt_buf + CRT_COLS, (CRT_SIZE - CRT_COLS) *
sizeof(uint16_t));
4         for (i = CRT_SIZE - CRT_COLS; i < CRT_SIZE; i++)
5             crt_buf[i] = 0x0700 | ' ';
6         crt_pos -= CRT_COLS;
7     }

```

这段代码是控制台屏幕滚动的实现。当屏幕内容写满时，需要将屏幕向上滚动一行，为新行腾出空间。

3. For the following questions you might wish to consult the notes for Lecture 2. These notes cover GCC's calling convention on the x86. Trace the execution of the following code step-by-step:

```
int x = 1, y = 3, z = 4;
cprintf("x %d, y %x, z %d\n", x, y, z);
```

In the call to `cprintf()`, to what does `fmt` point? To what does `ap` point? List (in order of execution) each call to `cons_putc`, `va_arg`, and `vcprintf`. For `cons_putc`, list its argument as well. For `va_arg`, list what `ap` points to before and after the call. For `vcprintf` list the values of its two arguments.

`fmt` -> "x %d, y %x, z %d\n"

`ap` -> 第一个参数的地址

可变参数被连续地存储在栈上，可以从小到大增大索引访问到它们

存储的顺序为从最右边的参数开始压栈

```
高地址
z = 4
y = 3
x = 1 <- ap
fmt = "x %d, y %x, z %d\n"
低地址
```

接下来按顺序分析`cons_putc`, `va_arg`和`vcprintf`的调用

```
cprintf("x %d, y %x, z %d\n", x, y, z)
->
vcprintf("x %d, y %x, z %d\n", (char *)&fmt + sizeof(fmt)) // ap指向第一个参数x
->
vprintfmt((void*)putc, &cnt, fmt, ap)
->
putc('x', &cnt)
->
cputchar('x') // 120为'x'的ASCII码值
->
cons_putc('x')
->
相同步骤cons_putc(' ')
->
// (ch = *(unsigned char *) fmt++) == '%'
// case 'd'
getint(&ap, lflag=0)
->
va_arg(*ap, int)
// before: ap-> (x=1)
// after: ap-> (y=3)
->
printnum(putc, putdat, num=1, base=10, width=-1, padc=' ')
->
putc('1', putdat)
```

```
->
cons_putc('1')
->
类似地打印后续字符串
```

4. Run the following code.

```
unsigned int i = 0x00646c72;
cprintf("H%x Wo%s", 57616, &i);
```

What is the output? Explain how this output is arrived at in the step-by-step manner of the previous exercise. Here's an ASCII table that maps bytes to characters. The output depends on that fact that the x86 is little-endian. If the x86 were instead big-endian what would you set `i` to in order to yield the same output? Would you need to change 57616 to a different value?

在 `kern/monitor.c` 中新增以下代码

```
int
mon_ans(int argc, char **argv, struct Trapframe *tf)
{
    unsigned int i = 0x00646c72;
    cprintf("H%x Wo%s", 57616, &i);
    return 0;
}
static struct Command commands[] = {
    { "testprintf", "Test cprintf", mon_testprintf },
};
```

```
K> ans
He110 World
```

因为 57616 的十六进制为 0xe110

`r`, `l`, `d` 的 ASCII 码分别为 0x72, 0x6C, 0x64

所以按 `%x` 方式解析 57616 时输出 e110

按 `%s` 方式解析 0x00646c72 (小端法) 时输出 rld

5. In the following code, what is going to be printed after 'y='? (note: the answer is not a specific value.) Why does this happen?

```
cprintf("x=%d y=%d", 3);
```

在输出到第2个%d时，`getuint`函数读取`ap`所指向的4个字节作为`num`

但只传入了1个可变参数，此时`ap`指向参数3之后的地址，包含随机的值

将其作为十六进制解析后输出，就是`y=`之后的输出

- Let's say that GCC changed its calling convention so that it pushed arguments on the stack in declaration order, so that the last argument is pushed last. How would you have to change `cprintf` or its interface so that it would still be possible to pass it a variable number of arguments?

颠倒参数的压栈顺序，变为从左往右，应该修改`va_start`设置`ap`为第1个参数位置(`ap`) = (`va_list`)
(`((char*)&fmt - sizeof(fmt))`), 修改`va_arg`取出下1个参数的方式为减小地址

Exercise 9

Determine where the kernel initializes its stack, and exactly where in memory its stack is located. How does the kernel reserve space for its stack? And at which "end" of this reserved area is the stack pointer initialized to point to?

```
kern/entry.S

movl    $(bootstacktop),%esp
->
    obj/kern/kernel.asm

f0100034:    bc 00 b0 10 f0            mov    $0xf010b000,%esp

.data
#####
# boot stack
#####
    .p2align    PGSHIFT    # force page alignment
    .globl      bootstack
bootstack:
    .space      KSTKSIZE
    .globl      bootstacktop
bootstacktop:
```

由反汇编可以看出栈顶位于`$0xf010b000`，`%esp`被设置为这个值，栈向低地址生长

内核通过`.space`在数据段中`bootstack`位置处初始化了`KSTKSIZE`这么多的空间

```
inc/memlayout.h

// All physical memory mapped at this address
#define KERNBASE    0xF0000000

// Kernel stack.
#define KSTACKTOP    KERNBASE
#define KSTKSIZE    (8*PGSIZE)    // size of a kernel stack
```

```
#define KSTKGAP      (8*PGSIZE)          // size of a kernel stack guard

inc/mmu.h

#define PGSIZE      4096                // bytes mapped by a page
```

Exercise 10

To become familiar with the C calling conventions on the x86, find the address of the `test_backtrace` function in `obj/kern/kernel.asm`, set a breakpoint there, and examine what happens each time it gets called after the kernel starts. How many 32-bit words does each recursive nesting level of `test_backtrace` push on the stack, and what are those words?

Note that, for this exercise to work properly, you should be using the patched version of QEMU available on the tools page or on Athena. Otherwise, you'll have to manually translate all breakpoint and memory addresses to linear addresses.

```
obj/kern/kernel.asm

f0100040 <test_backtrace>:
```

```
(gdb) b test_backtrace
Breakpoint 1 at 0xf0100040
(gdb) c
Continuing.
The target architecture is set to "i386".
=> 0xf0100040 <test_backtrace>: push    %ebp

Breakpoint 1, 0xf0100040 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf01000f4 0x00000005 0x00001aac 0x00000660
0xf010afe8: 0x00000000 0x00000000 0x00010094 0x00000000
(gdb) si
=> 0xf0100041 <test_backtrace+1>: mov    %esp,%ebp
0xf0100041 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf010aff8 0xf01000f4 0x00000005 0x00001aac
0xf010afe8: 0x00000660 0x00000000 0x00000000 0x00010094
(gdb) si
=> 0xf0100043 <test_backtrace+3>: push   %esi
0xf0100043 in test_backtrace ()
(gdb) x/8wx $esp
0xf010afd8: 0xf010aff8 0xf01000f4 0x00000005 0x00001aac
0xf010afe8: 0x00000660 0x00000000 0x00000000 0x00010094
(gdb) si
=> 0xf0100044 <test_backtrace+4>: push   %ebx
0xf0100044 in test_backtrace ()
(gdb) x/8wx $esp
```

0xf010afd4:	0x00010094	0xf010aff8	0xf01000f4	0x00000005
0xf010afe4:	0x00001aac	0x00000660	0x00000000	0x00000000

```

0xf010afbc: 0xf0100076    <- ESP指向这里 (递归返回地址)
0xf010afc0: 0x00000004
0xf010afc4: 0x00000005
0xf010afc8: 0x00000000
0xf010afcc: 0xf010004a
0xf010afd0: 0xf010c308
0xf010afd4: 0x00010094
0xf010afd8: 0xf010aff8    <- push    %ebp
0xf010afdc: 0xf01000f4    <- 压入返回地址i386_init到旧的ESP
0xf010afe0: 0x00000005    <- 参数 x=5 (第一层)
0xf010afe4: 0x00001aac
0xf010afe8: 0x00000660
0xf010afec: 0x00000000
0xf010aff0: 0x00000000
0xf010aff4: 0x00010094
0xf010aff8: 0x00000000    <- 第一层的EBP (栈底)

```

Exercise 11

Implement the backtrace function as specified above. Use the same format as in the example, since otherwise the grading script will be confused. When you think you have it working right, run `make grade` to see if its output conforms to what our grading script expects, and fix it if it doesn't. After you have handed in your Lab 1 code, you are welcome to change the output format of the backtrace function any way you like.

If you use `read_ebp()`, note that GCC may generate "optimized" code that calls `read_ebp()` before `mon_backtrace()`'s function prologue, which results in an incomplete stack trace (the stack frame of the most recent function call is missing). While we have tried to disable optimizations that cause this reordering, you may want to examine the assembly of `mon_backtrace()` and make sure the call to `read_ebp()` is happening after the function prologue.

```

int
mon_backtrace(int argc, char **argv, struct Trapframe *tf)
{
    uint32_t *ebp = (uint32_t *)read_ebp();
    while (ebp != 0) {
        uint32_t eip = *(ebp + 1);
        cprintf("ebp %08x eip %08x args %08x %08x %08x %08x\n", ebp, eip, *
(ebp + 2), *(ebp + 3), *(ebp + 4), *(ebp + 5), *(ebp + 6));
        ebp = (uint32_t *)(*ebp);
    }
    return 0;
}

```

Exercise 12

Modify your stack backtrace function to display, for each eip, the function name, source file name, and line number corresponding to that eip.

In `debuginfo_eip`, where do `_STAB*` come from? This question has a long answer; to help you to discover the answer, here are some things you might want to do:

look in the file `kern/kernel.ld` for `_STAB*` run `objdump -h obj/kern/kernel` run `objdump -G obj/kern/kernel` run `gcc -pipe -nostdinc -O2 -fno-builtin -l -MD -Wall -Wno-format -DJOS_KERNEL -gstabs -c -S kern/init.c`, and look at `init.s`. see if the bootloader loads the symbol table in memory as part of loading the kernel binary Complete the implementation of `debuginfo_eip` by inserting the call to `stab_binsearch` to find the line number for an address.

Add a backtrace command to the kernel monitor, and extend your implementation of `mon_backtrace` to call `debuginfo_eip` and print a line for each stack frame of the form:

```
K> backtrace
Stack backtrace:
  ebp f010ff78  eip f01008ae  args 00000001 f010ff8c 00000000 f0110580 00000000
      kern/monitor.c:143: monitor+106
  ebp f010ffd8  eip f0100193  args 00000000 00001aac 00000660 00000000 00000000
      kern/init.c:49: i386_init+59
  ebp f010fff8  eip f010003d  args 00000000 00000000 0000ffff 10cf9a00 0000ffff
      kern/entry.S:70: <unknown>+0
K>
```

Each line gives the file name and line within that file of the stack frame's eip, followed by the name of the function and the offset of the eip from the first instruction of the function (e.g., `monitor+106` means the return eip is 106 bytes past the beginning of `monitor`).

Be sure to print the file and function names on a separate line, to avoid confusing the grading script.

Tip: `printf` format strings provide an easy, albeit obscure, way to print non-null-terminated strings like those in STABS tables. `printf("%.s", length, string)` prints at most `length` characters of `string`. Take a look at the `printf` man page to find out why this works.

You may find that some functions are missing from the backtrace. For example, you will probably see a call to `monitor()` but not to `runcmd()`. This is because the compiler in-lines some function calls. Other optimizations may cause you to see unexpected line numbers. If you get rid of the `-O2` from `GNUMakefile`, the backtraces may make more sense (but your kernel will run more slowly).

```
int
mon_backtrace(int argc, char **argv, struct Trapframe *tf)
{
    uint32_t *ebp = (uint32_t *)read_ebp();
    while (ebp != 0) {
        //打印ebp, eip, 最近的五个参数
        uint32_t eip = *(ebp + 1);
```

```

    cprintf("ebp %08x eip %08x args %08x %08x %08x %08x %08x\n", ebp, eip, *
(ebp + 2), *(ebp + 3), *(ebp + 4), *(ebp + 5), *(ebp + 6));

    struct Eipdebuginfo info;
    int ret = debuginfo_eip((uintptr_t)eip, &info);
    // cprintf("debuginfo_eip returned: %d\n", ret);
    cprintf("\t%s:%d: %.*s+%d\n", info.eip_file, info.eip_line,
info.eip_fn_namelen, info.eip_fn_name, eip-info.eip_fn_addr);
    ebp = (uint32_t *)(*ebp);
}
return 0;
}

```

```

static void
stab_binsearch(const struct Stab *stabs, int *region_left, int *region_right,
               int type, uintptr_t addr)
{
    ...
    stab_binsearch(stabs, &lline, &rline, N_SLINE, addr);
    if (lline <= rline) {
        info->eip_line = stabs[lline].n_desc;
    } else {
        info->eip_line = 0;
        return -1;
    }
    ...
}

```

```

static struct Command commands[] = {
    { "help", "Display this list of commands", mon_help },
    { "kerninfo", "Display information about the kernel", mon_kerninfo },
    { "backtrace", "Display a backtrace of the function stack", mon_backtrace },
};

```